SKIN CANCER DETECTION

From CNNs to Swin Transformers

Team: TENSOR_TITANS

Akshat Bhatt (202301460)
Jiten Bharga (202301466)

Course: IT-549 Deep Learning
Assigned by: Prof. Arpit Rana

Contents

1 Introduction **3**
- 1.1 The Problem Landscape 3
- 1.2 Problem Statement 4

2 The HAM10000 Dataset and Molecular Biology **5**
- 2.1 Dataset Overview 5
- 2.2 Molecular Pathways of Skin Lesions 5
  - 2.2.1 Actinic Keratosis / Intraepithelial Carcinoma (`akiec`) 5
  - 2.2.2 Basal Cell Carcinoma (`bcc`) 6
  - 2.2.3 Malignant Melanoma (`mel`) 6

3 Convolutional Neural Networks: Deep Mathematical Foundations **7**
- 3.1 The Necessity of the Convolution Operator 7
- 3.2 Mathematical Formulation of 2D/3D Convolution 7
- 3.3 Dimensionality and Parameter Constraints 8
- 3.4 Backpropagation Through Convolutions 8
- 3.5 Activation Functions: The Calculus of ReLU 8

4 ResNet-50: Solving the Degradation Problem **10**
- 4.1 Mathematical Formulation of the Degradation Problem 10
- 4.2 The Residual Block Mapping 10
- 4.3 Calculus of the Shortcut Connections 10

5 EfficientNet: Deep Mathematical Scaling and MBConv **12**
- 5.1 The Optimization Problem of Network Scaling 12
- 5.2 Mobile Inverted Bottleneck Convolution (MBConv) 13
- 5.3 Squeeze-and-Excitation (SE) Network Mathematics 14

6 YOLOv4: Regression-Based Object Detection **16**
- 6.1 Reformulating Classification as Bounding Box Regression 16
- 6.2 The Complete IoU (CIoU) Loss Formulation 16
- 6.3 Total YOLO Loss Function 17

7 Vision Transformers (ViT): Deep Mathematical Foundations **18**
- 7.1 Image Patch Extraction and Embedding 18
- 7.2 The Mathematics of Scaled Dot-Product Attention 18
- 7.3 Multi-Head Attention (MSA) and the MLP Block 19
- 7.4 Complexity Analysis of ViT 20

8 Swin Transformer: Hierarchical Shifted-Window Math **21**
8.1 Window-Based Self Attention (W-MSA) Complexity 21
8.2 Shifted Window Partitioning Mathematics 21
8.3 Efficient Batch Computation via Cyclic Shift 21
8.4 Relative Position Bias Parameterization 22

9 Our Methodology & Results **23**
9.1 Loss Formulation: Standard Cross-Entropy 23
9.2 Two-Phase Fine-Tuning: Linear Algebraic Formulation 23
9.3 Optimization Dynamics: AdamW with Decoupled Weight Decay 24

10 Conclusion **25**

11 Datasets and Repositories **26**

12 References **27**

1 INTRODUCTION

1.1 The Problem Landscape

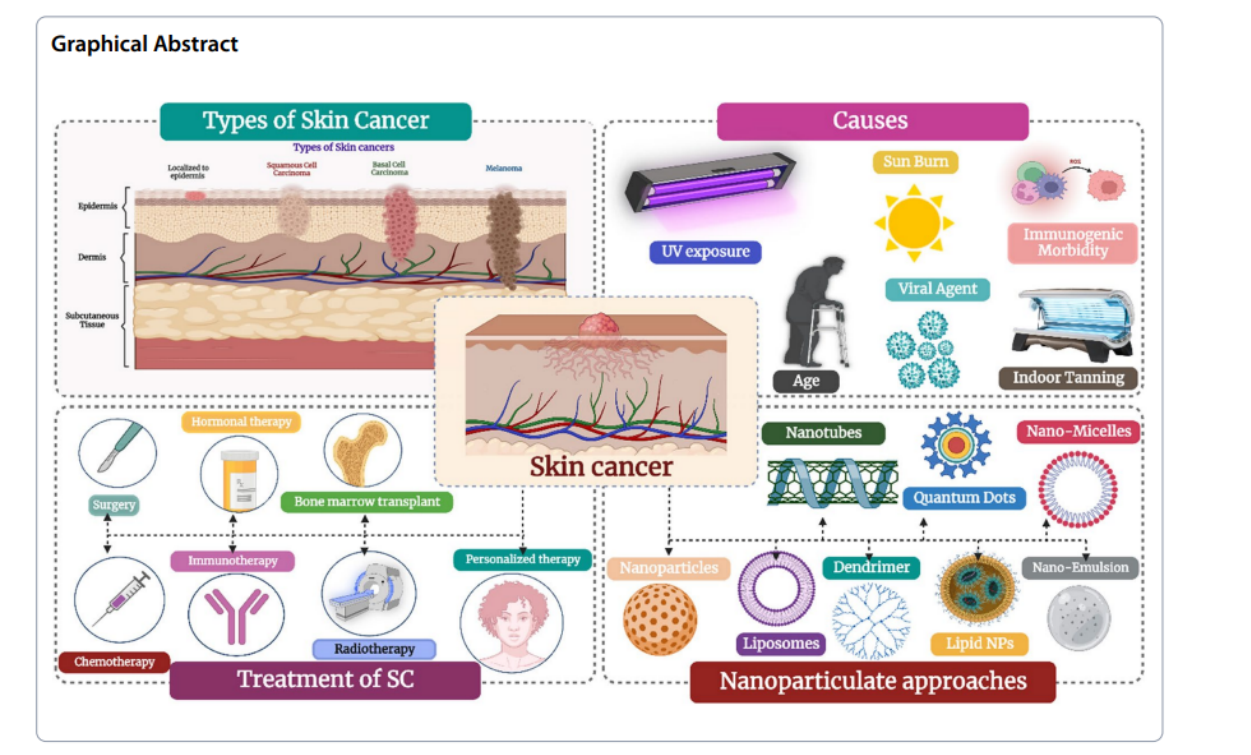
Skin cancer is the most prevalent malignancy worldwide, with incidence rates continuing to rise exponentially each decade. Over 10,000 new cases of melanoma are diagnosed daily globally. Non-melanoma skin cancers (basal cell carcinoma and squamous cell carcinoma) account for more than 95% of all skin cancer diagnoses. Early diagnosis dramatically improves prognosis; however, manual dermoscopic analysis is time-consuming, requires expert training, and is subject to severe inter-observer variability.

Key Epidemiological Facts

Melanoma alone is responsible for approximately 75% of all skin cancer-related deaths despite constituting a small fraction of total cases. Immunosuppressed patients (organ transplant recipients, HIV-positive individuals) face a 65 to 250-fold elevated risk of developing squamous cell carcinoma from precancerous lesions.

Key Risk Drivers:

- **UV exposure**: UV-A at 320–400 nm; UV-B at 290–320 nm remains the primary modifiable risk factor, causing direct DNA damage and oxidative stress.
- **Ageing populations**: Increases cumulative cancer risk due to somatic mutation accumulation over successive cell generations.
- **Immunosuppression**: Transplant recipients and HIV patients raise skin cancer incidence dramatically.
- **Phenotypic susceptibility**: Fair skin, freckling, and multiple melanocytic nevi.



1.2 Problem Statement

The core challenge addressed in this work is formulated as follows:

Research Question

How do we achieve automated, mathematically rigorous, and highly accurate classification of eight distinct skin lesion types—including rare malignancies—while mitigating severe class imbalance, providing clinically meaningful predictions, and remaining within real-time operational computational constraints?

Operational Constraints:

- Must process a dermoscopic image in under 2 seconds (clinical acceptability threshold).
- Must correctly identify melanoma (`mel`) and basal cell carcinoma (`bcc`) with high sensitivity to minimise false negatives.
- Must generalise across varied skin tones, lesion sizes, and imaging devices.
- Must handle extreme class imbalance (the dominant class `nv` constitutes ≈67% of samples).

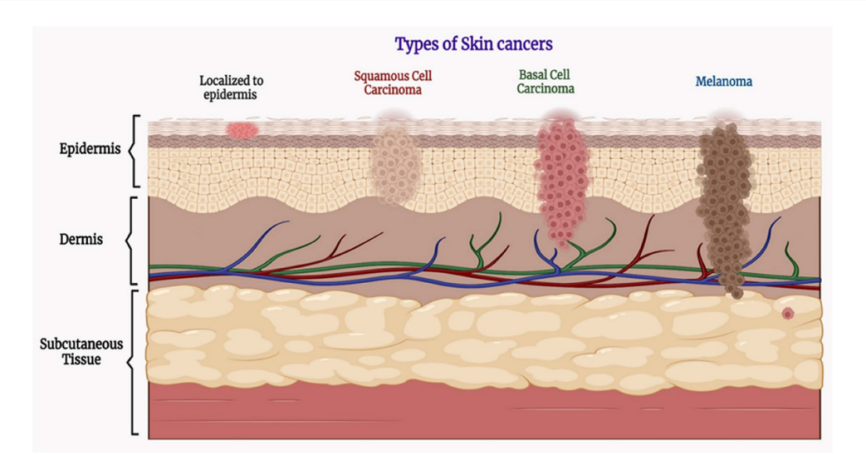
2 THE HAM10000 DATASET AND MOLECULAR BIOLOGY

2.1 Dataset Overview

The **HAM10000** (*Human Against Machine with 10,000 training images*) dataset contains 10,015 dermoscopic images spanning seven diagnostic classes. The dataset was collected over 20 years from the Medical University of Vienna and Melanoma Institute Australia.

Table 1: HAM10000 Class Distribution

Code	Full Lesion Name	Malignant?	Count	%
`akiec`	Actinic keratosis / Intraepithelial carcinoma	Precancerous	327	3.3%
`bcc`	Basal cell carcinoma	Yes	514	5.1%
`bkl`	Benign keratosis (seborrhoeic, solar lentigo)	No	1,099	11.0%
`df`	Dermatofibroma	No	115	1.1%
`mel`	Malignant melanoma	Yes	1,113	11.1%
`nv`	Melanocytic nevus (common mole)	No	6,705	66.9%
`vasc`	Vascular skin lesions (angiomas, granulomas)	No	142	1.4%



2.2 Molecular Pathways of Skin Lesions

2.2.1 Actinic Keratosis / Intraepithelial Carcinoma (`akiec`)

Actinic keratosis arises from chronic cumulative UV exposure. UV-B irradiation (290–320 nm) penetrates to the epidermal basal layer, causing **thymine dimers** (cyclobutane pyrimidine dimers, CPDs) through the covalent bonding of adjacent thymine bases.

The primary molecular pathway involves C→T transitions at dipyrimidine sites in the **TP53** tumour suppressor gene. Loss of p53 function removes the “guardian of the genome”, allowing mutated keratinocytes to escape apoptosis and proliferate.

2.2.2 Basal Cell Carcinoma (`bcc`)

BCC is almost exclusively driven by mutations in the **Hedgehog signalling pathway**. Normally, the PTCH1 (Patched 1) receptor constitutively inhibits SMO (Smoothened). When the Hedgehog ligand (SHH) binds PTCH1, it releases SMO inhibition. In BCC, inactivating mutations of PTCH1 (found in $>$85% of sporadic BCCs) permanently remove the inhibition of SMO, leading to constitutive activation of GLI transcription factors and uncontrolled proliferation.

2.2.3 Malignant Melanoma (`mel`)

Melanoma arises from melanocytes. The most common driver is the **MAPK Pathway**.

MAPK Pathway Dynamics

The **BRAF V600E Mutation** is found in 40–60% of cutaneous melanomas. The signalling cascade is expressed as:

$$\text{Growth factor} \xrightarrow{binds} \text{RTK} \xrightarrow{activates} \text{RAS} \xrightarrow{phosphorylates} \text{BRAF}^{\text{V600E}} \xrightarrow{phosphorylates} \text{MEK1/2} \xrightarrow{phosphorylates} \text{ERK}$$

The mutation at valine 600 to glutamic acid mimics regulatory phosphorylation, rendering the kinase domain constitutively active. This results in a constant flux of downstream ERK1/2 into the nucleus, bypassing all upstream regulatory checkpoints.

PI3K/AKT Pathway Dynamics

A secondary driver is **PTEN Loss** (30–60% of non-familial melanomas). PTEN normally functions as a lipid phosphatase, converting Phosphatidylinositol (3,4,5)-trisphosphate (PIP_3) back to PIP_2:

$$\text{PIP}_3 \xrightarrow{\text{PTEN}} \text{PIP}_2 + \text{P}_i$$

Without PTEN, PIP_3 accumulates, chronically activating PDK1 and subsequently AKT, which drives mTOR-mediated cell survival and proliferation.

3 CONVOLUTIONAL NEURAL NETWORKS: DEEP MATHEMATICAL FOUNDATIONS

3.1 The Necessity of the Convolution Operator

A standard fully-connected Artificial Neural Network (ANN) connects every neuron in layer l to every neuron in layer $l+1$. Let the input image be represented as a tensor $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$. If we unroll $\mathbf{X}$ into a vector $\mathbf{x} \in \mathbb{R}^{HWC}$, the dense transformation to a hidden layer $\mathbf{h} \in \mathbb{R}^{M}$ is:

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \tag{1}$$

where $\mathbf{W} \in \mathbb{R}^{M \times (HWC)}$. For a modern high-resolution dermoscopic image ($448 \times 448 \times 3$), a single neuron requires $602,112$ weights. If we desire a hidden layer of size $M = 1000$, the weight matrix $\mathbf{W}$ contains over 6×10^8 parameters for just the *first layer*. This causes immediate intractability and severe overfitting.

CNNs solve this via **sparse interactions**, **parameter sharing**, and **equivariant representations**.

3.2 Mathematical Formulation of 2D/3D Convolution

Let $\mathbf{X} \in \mathbb{R}^{H \times W \times D_{\text{in}}}$ be the input tensor to a convolutional layer. We define a set of K learnable filters (kernels). The k-th filter is a tensor $\mathbf{W}^{(k)} \in \mathbb{R}^{F \times F \times D_{\text{in}}}$.

The discrete cross-correlation (commonly referred to as convolution in Deep Learning) to compute the output feature map $\mathbf{Z}^{(k)} \in \mathbb{R}^{H' \times W'}$ is strictly defined as:

The Convolution Integral (Discrete Cross-Correlation)

$$z_{i,j}^{(k)} = b^{(k)} + \sum_{c=1}^{D_{\text{in}}} \sum_{p=0}^{F-1} \sum_{q=0}^{F-1} W_{c,p,q}^{(k)} \cdot x_{c,\, i \cdot s+p-P,\, j \cdot s+q-P} \tag{2}$$

where:

- $i \in \{0, \ldots, H'-1\}$ and $j \in \{0, \ldots, W'-1\}$ are the spatial indices.
- s is the scalar stride of the convolution.
- P is the zero-padding applied to the boundaries of $\mathbf{X}$.
- $b^{(k)} \in \mathbb{R}$ is the learned bias term for the k-th filter.

3.3 Dimensionality and Parameter Constraints

The spatial dimensions of the output tensor $\mathbf{Z}$ are determined by the input size H, W, the filter size F, stride s, and padding P:

Spatial Output Topology

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} - F + 2P}{s} \right\rfloor + 1 \tag{3}$$

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} - F + 2P}{s} \right\rfloor + 1 \tag{4}$$

The total number of parameters in this layer, θ_{conv}, is drastically reduced compared to a dense layer:

$$\theta_{\text{conv}} = K \cdot (F \cdot F \cdot D_{\text{in}} + 1) \tag{5}$$

3.4 Backpropagation Through Convolutions

To optimize a CNN using Gradient Descent, we must compute the derivative of the loss function $\mathcal{L}$ with respect to the filter weights $\mathbf{W}$ and the input $\mathbf{X}$ (to pass the gradient to previous layers).

Let $\delta_{i,j}^{(k)} = \frac{\partial \mathcal{L}}{\partial z_{i,j}^{(k)}}$ be the upstream gradient passed down to this layer. The gradient with respect to a specific weight $W_{c,p,q}^{(k)}$ is:

Convolutional Weight Gradient

$$\frac{\partial \mathcal{L}}{\partial W_{c,p,q}^{(k)}} = \sum_{i=0}^{H'-1} \sum_{j=0}^{W'-1} \delta_{i,j}^{(k)} \cdot x_{c,\, i \cdot s+p-P,\, j \cdot s+q-P} \tag{6}$$

Remarkably, the gradient with respect to the weights is itself a valid convolution operation between the upstream gradient δ and the input tensor $\mathbf{X}$.

3.5 Activation Functions: The Calculus of ReLU

Following the affine transformation, a non-linear activation function is applied. We eschew the historically prevalent sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$ due to the **vanishing gradient problem**.

Consider the derivative of the sigmoid function:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)) \tag{7}$$

The maximum value of $\sigma'(x)$ is 0.25 (at $x = 0$). By the chain rule, a deep network of L layers will

multiply these gradients:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} \propto \prod_{l=1}^{L} \sigma'(z^{(l)}) \leq (0.25)^L \tag{8}$$

As L increases, the gradient shrinks exponentially toward zero, preventing the early layers from updating.

Rectified Linear Unit (ReLU)

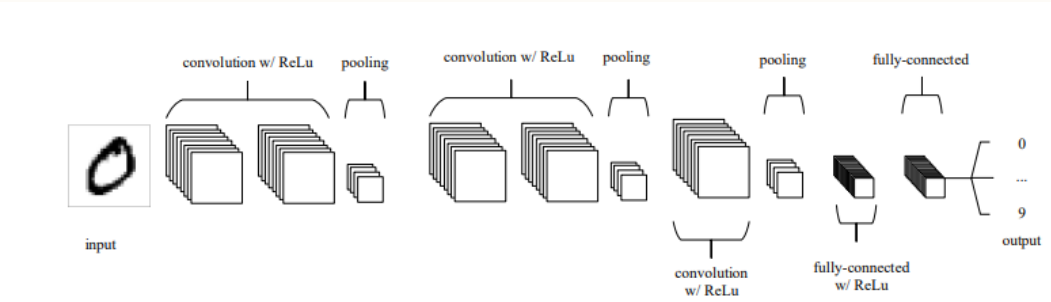
To solve this, modern networks utilize ReLU:

$$\text{ReLU}(x) = \max(0,\, x) \tag{9}$$

Its derivative is a simple Heaviside step function:

$$\frac{d}{dx}\text{ReLU}(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x < 0 \end{cases} \tag{10}$$

Because the derivative is exactly 1 for all positive inputs, the gradient flows unattenuated through arbitrarily deep networks, stabilizing optimization.



4 RESNET-50: SOLVING THE DEGRADATION PROBLEM

4.1 Mathematical Formulation of the Degradation Problem

Let $\mathcal{H}(\mathbf{x})$ be the optimal underlying mapping to be fit by a few stacked layers, with $\mathbf{x}$ denoting the inputs to the first of these layers. If we define an arbitrarily deep network, we can hypothesize that the optimal solution is simply the shallower network's output, with all subsequent added layers performing an identity mapping: $\mathcal{H}(\mathbf{x}) = \mathbf{x}$.

However, empirical evidence by He et al. (2016) showed that optimizing multiple non-linear layers (e.g., stacked convolutions and ReLUs) to approximate the identity function $I(\mathbf{x}) = \mathbf{x}$ is highly ill-conditioned. The optimizer fails, causing training error to *increase* as depth increases.

4.2 The Residual Block Mapping

ResNet fundamentally alters the topology. Instead of expecting stacked layers to approximate $\mathcal{H}(\mathbf{x})$ directly, we explicitly let these layers approximate a **residual function**:

Residual Mapping Formulation

$$\mathcal{F}(\mathbf{x}, \{W_i\}) := \mathcal{H}(\mathbf{x}) - \mathbf{x} \tag{11}$$

The output of the block $\mathbf{y}$ is then defined as:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x} \tag{12}$$

where the addition is performed element-wise.

If the optimal mapping is closer to identity than to a zero mapping, the solver simply drives the weights of $\mathcal{F}$ to zero: $\mathcal{F}(\mathbf{x}) \to 0$, leaving $\mathbf{y} = \mathbf{x}$.

4.3 Calculus of the Shortcut Connections

Consider a network composed of L sequential residual blocks. The output of the l-th block is:

$$\mathbf{x}_{l+1} = \mathbf{x}_l + \mathcal{F}(\mathbf{x}_l, \mathcal{W}_l) \tag{13}$$

By recursion, the feature vector at any deeper layer L can be expressed directly as the sum of the feature vector at shallower layer l plus the sum of all intermediate residual functions:

$$\mathbf{x}_L = \mathbf{x}_l + \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i, \mathcal{W}_i) \tag{14}$$

Applying the chain rule, the gradient of the loss $\mathcal{L}$ with respect to the activation at layer l is:

Unattenuated Gradient Flow

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} \frac{\partial \mathbf{x}_L}{\partial \mathbf{x}_l} \tag{15}$$

$$= \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} \left(1 + \frac{\partial}{\partial \mathbf{x}_l} \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i, \mathcal{W}_i) \right) \tag{16}$$

$$= \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} + \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} \sum_{i=l}^{L-1} \frac{\partial \mathcal{F}(\mathbf{x}_i, \mathcal{W}_i)}{\partial \mathbf{x}_l} \tag{17}$$

This equation reveals the brilliance of the architecture: the gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{x}_l}$ is decomposed into two additive terms. The first term, $\frac{\partial \mathcal{L}}{\partial \mathbf{x}_L}$, propagates information *directly* back to shallower layers without any weight matrix multiplication. This guarantees that the gradient will never vanish to absolute zero, even if the mini-batch gradients in the second term become infinitesimally small.

5 EFFICIENTNET: DEEP MATHEMATICAL SCALING AND MBCONV

5.1 The Optimization Problem of Network Scaling

Let a Convolutional Neural Network $\mathcal{N}$ be defined as a sequence of composed transformations. We can mathematically define the network as a function of its depth d, width w, and input resolution r:

$$\mathcal{N}(d, w, r) = \bigodot_{i=1\ldots S} \mathcal{F}_i^{d\cdot\hat{L}_i}\left(X_{\langle r\cdot\hat{H}_i,\, r\cdot\hat{W}_i,\, w\cdot\hat{C}_i\rangle}\right) \tag{18}$$

Where $\mathcal{F}_i$ is the layer transformation in stage i, $\hat{L}_i$ is the baseline number of layers, and $\langle\hat{H}_i, \hat{W}_i, \hat{C}_i\rangle$ denotes the baseline tensor shape.

The objective is to maximize model accuracy for any given resource budget. This is a constrained optimization problem:

Compound Scaling Optimization

$$\max_{d,w,r} \quad \text{Accuracy}(\mathcal{N}(d, w, r)) \tag{19}$$

$$\text{subject to} \quad \mathcal{N}(d, w, r) = \bigodot_{i=1\ldots S} \mathcal{F}_i^{d\cdot\hat{L}_i}\left(X_{\langle r\cdot\hat{H}_i, r\cdot\hat{W}_i, w\cdot\hat{C}_i\rangle}\right) \tag{20}$$

$$\text{Memory}(\mathcal{N}) \leq \text{target_memory} \tag{21}$$

$$\text{FLOPs}(\mathcal{N}) \leq \text{target_flops} \tag{22}$$

Because convolution operations dominate the FLOPs, we can analyze how FLOPs scale. For a standard convolution, FLOPs $\propto d \cdot w^2 \cdot r^2$. Depth d scales linearly, but width w and resolution r scale quadratically (doubling resolution quadruples the number of pixels; doubling width quadruples the $C_{\text{in}} \times C_{\text{out}}$ cross-channel calculations).

To constrain the scaling, Tan & Le introduced the **Compound Coefficient** ϕ:

Compound Coefficient Constraints

$$d = \alpha^\phi \tag{23}$$

$$w = \beta^\phi \tag{24}$$

$$r = \gamma^\phi \tag{25}$$

$$s.t. \quad \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2 \tag{26}$$

$$\alpha \geq 1,\ \beta \geq 1,\ \gamma \geq 1 \tag{27}$$

Under these constraints, scaling the network by a factor of ϕ increases the total computational cost by exactly 2^ϕ. For EfficientNet-B0, grid search yielded optimal constants: $\alpha = 1.2$, $\beta = 1.1$, $\gamma = 1.15$.

5.2 Mobile Inverted Bottleneck Convolution (MBConv)

EfficientNet utilizes the MBConv block, relying heavily on **Depthwise Separable Convolutions**. Let us rigorously analyze the computational savings of this operation.

Consider an input tensor $\mathbf{X} \in \mathbb{R}^{H \times W \times C_{in}}$ and a standard convolution with a kernel $\mathbf{K} \in \mathbb{R}^{F \times F \times C_{in} \times C_{out}}$. The number of floating-point operations (FLOPs) required to compute the output $\mathbf{Y} \in \mathbb{R}^{H \times W \times C_{out}}$ (assuming stride 1, padding same) is:

$$\text{FLOPs}_{\text{standard}} = F \cdot F \cdot C_{in} \cdot C_{out} \cdot H \cdot W \tag{28}$$

A Depthwise Separable Convolution factors this into two distinct steps:

1. **Depthwise Convolution**: Applies a single $F \times F$ spatial filter to each input channel independently.
2. **Pointwise Convolution**: Applies a $1 \times 1 \times C_{in} \times C_{out}$ filter to linearly combine the depthwise outputs across channels.

The FLOPs for the separated operations are:

$$\text{FLOPs}_{\text{depthwise}} = F \cdot F \cdot C_{in} \cdot 1 \cdot H \cdot W \tag{29}$$

$$\text{FLOPs}_{\text{pointwise}} = 1 \cdot 1 \cdot C_{in} \cdot C_{out} \cdot H \cdot W \tag{30}$$

$$\text{FLOPs}_{\text{separable}} = (F^2 \cdot C_{in} + C_{in} \cdot C_{out}) \cdot H \cdot W \tag{31}$$

We can now compute the computational reduction ratio $\mathcal{R}$:

Depthwise Separable Efficiency Ratio

$$\mathcal{R} = \frac{\text{FLOPs}_{\text{separable}}}{\text{FLOPs}_{\text{standard}}} \tag{32}$$

$$= \frac{(F^2 \cdot C_{in} + C_{in} \cdot C_{out}) \cdot H \cdot W}{F^2 \cdot C_{in} \cdot C_{out} \cdot H \cdot W} \tag{33}$$

$$= \frac{F^2 + C_{out}}{F^2 \cdot C_{out}} \tag{34}$$

$$= \frac{1}{C_{out}} + \frac{1}{F^2} \tag{35}$$

For a typical 3×3 kernel ($F = 3$) and a moderate number of output channels (e.g., $C_{out} = 128$),

the ratio is: $\mathcal{R} = \frac{1}{128} + \frac{1}{9} \approx 0.0078 + 0.1111 = 0.1189$. Thus, the MBConv block requires roughly $\sim 11.9\%$ of the computation of a standard convolution layer while maintaining similar expressive capacity.

5.3 Squeeze-and-Excitation (SE) Network Mathematics

The SE block introduces channel-wise attention to the MBConv block. Let the input feature map be $\mathbf{U} \in \mathbb{R}^{H \times W \times C}$.

1. Squeeze Operation: We aggregate the global spatial information into a channel descriptor $\mathbf{z} \in \mathbb{R}^C$ using Global Average Pooling. The c-th element of $\mathbf{z}$ is:

$$z_c = \mathbf{F}_{sq}(\mathbf{u}_c) = \frac{1}{H \times W} \sum_{i=1}^{H} \sum_{j=1}^{W} u_c(i, j) \tag{36}$$

2. Excitation Operation: We capture channel-wise dependencies via a two-layer bottleneck multi-layer perceptron (MLP). Let $\mathbf{W}_1 \in \mathbb{R}^{\frac{C}{r} \times C}$ be the dimensionality reduction matrix (where r is the reduction ratio), and $\mathbf{W}_2 \in \mathbb{R}^{C \times \frac{C}{r}}$ be the dimensionality restoration matrix. The excitation vector $\mathbf{s} \in \mathbb{R}^C$ is:

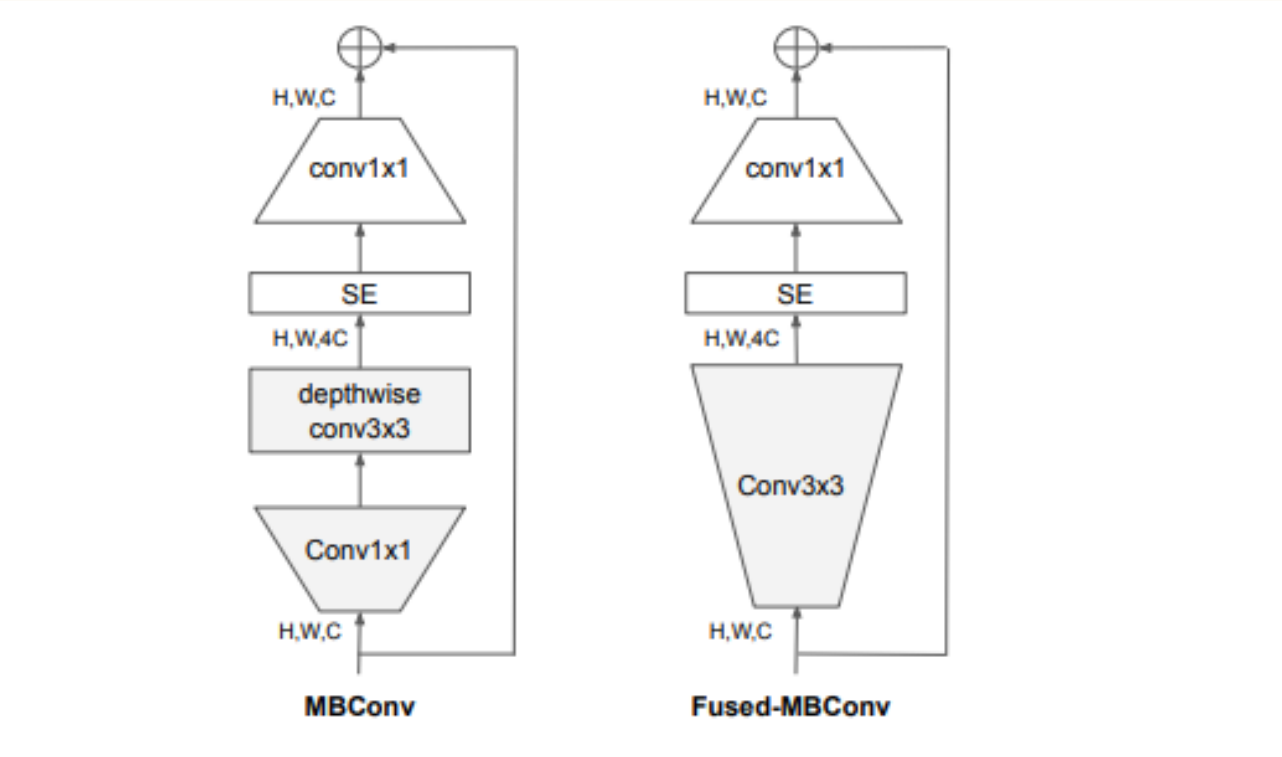
$$\mathbf{s} = \mathbf{F}_{ex}(\mathbf{z}, \mathbf{W}) = \sigma(\mathbf{W}_2 \delta(\mathbf{W}_1 \mathbf{z})) \tag{37}$$

where δ refers to the ReLU function and σ refers to the Sigmoid function mapping values to $(0, 1)$.

3. Scaling: The final output of the block $\widetilde{\mathbf{X}}$ is obtained by scaling the input feature map $\mathbf{U}$ by the activations $\mathbf{s}$:

$$\widetilde{\mathbf{x}}_c = \mathbf{F}_{scale}(\mathbf{u}_c, s_c) = s_c \cdot \mathbf{u}_c \tag{38}$$

This mathematical framework allows the network to dynamically suppress non-informative channels (e.g., background skin noise) and amplify diagnostically relevant channels (e.g., contrast representing an irregular border).


H,W,C
conv1x1
SE
H,W,4C
depthwise
conv3x3
Conv1x1
H,W,C
MBConv
H,W,C
conv1x1
SE
H,W,4C
Conv3x3
H,W,C
Fused-MBConv

6 YOLOV4: REGRESSION-BASED OBJECT DETECTION

6.1 Reformulating Classification as Bounding Box Regression

YOLO (You Only Look Once) reframes the analysis from pure whole-image classification to a spatial regression task. The image is divided into an $S \times S$ grid. If the center of an object falls into a grid cell, that cell is "responsible" for detecting that object.

Each grid cell predicts B bounding boxes and confidence scores for those boxes. A bounding box is mathematically parameterized by five values: $(x, y, w, h, \text{conf})$.

- $x, y \in [0, 1]$ represent the center coordinates of the box relative to the bounds of the grid cell.
- $w, h \in [0, 1]$ represent the width and height relative to the entire image dimensions.
- conf represents the predicted Intersection over Union (IoU) between the predicted box and the ground truth box.

6.2 The Complete IoU (CIoU) Loss Formulation

In YOLOv4, bounding box regression is optimized using the **Complete Intersection over Union (CIoU)** loss. Standard IoU simply measures overlap:

$$\text{IoU} = \frac{|\mathbf{B} \cap \mathbf{B}^{gt}|}{|\mathbf{B} \cup \mathbf{B}^{gt}|} \tag{39}$$

However, if there is no overlap, IoU is exactly zero, and the gradient becomes zero, halting learning.

CIoU solves this by incorporating three geometric factors: overlap area, central point distance, and aspect ratio consistency.

CIoU Loss Function

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v \tag{40}$$

Where the components are defined as:

- $\mathbf{b}$ and $\mathbf{b}^{gt}$ are the central points of the predicted and ground truth boxes.
- $\rho(\cdot)$ is the Euclidean distance.
- c is the diagonal length of the smallest enclosing box covering both bounding boxes.
- v measures the consistency of the aspect ratio:

$$v = \frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w}{h}\right)^2 \tag{41}$$

- α is a positive trade-off parameter:

$$\alpha = \frac{v}{(1 - \text{IoU}) + v} \tag{42}$$

6.3 Total YOLO Loss Function

The total loss $\mathcal{L}_{YOLO}$ combines the CIoU bounding box loss, the objectness confidence loss, and the classification loss. Let $\mathbf{1}_{ij}^{obj}$ be an indicator variable equal to 1 if the j-th bounding box predictor in cell i is responsible for the object, and 0 otherwise.

$$\begin{aligned}
\mathcal{L}_{YOLO} = \lambda_{coord} &\sum_{i=0}^{S^2}\sum_{j=0}^{B} \mathbf{1}_{ij}^{obj} \mathcal{L}_{CIoU}(\mathbf{b}_{ij}, \mathbf{b}_{ij}^{gt}) \\
&- \sum_{i=0}^{S^2}\sum_{j=0}^{B} \mathbf{1}_{ij}^{obj}\left[\hat{C}_i \log(C_i) + (1 - \hat{C}_i)\log(1 - C_i)\right] \\
&- \lambda_{noobj} \sum_{i=0}^{S^2}\sum_{j=0}^{B} \mathbf{1}_{ij}^{noobj}\left[\hat{C}_i \log(C_i) + (1 - \hat{C}_i)\log(1 - C_i)\right] \\
&- \sum_{i=0}^{S^2} \mathbf{1}_{i}^{obj} \sum_{c \in classes}\left[\hat{p}_i(c)\log(p_i(c)) + (1 - \hat{p}_i(c))\log(1 - p_i(c))\right]
\end{aligned} \tag{43}$$

Here, C_i represents the objectness score, and $p_i(c)$ represents the class probability. Binary Cross-Entropy is used for classification to support multi-label predictions if necessary.

7 VISION TRANSFORMERS (VIT): DEEP MATHEMATICAL FOUNDATIONS

7.1 Image Patch Extraction and Embedding

The Vision Transformer (Dosovitskiy et al., 2021) bypasses convolutions entirely, treating image processing as a sequence-to-sequence translation task.

Given an image $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$, we partition it into a sequence of flattened 2D patches $\mathbf{x}_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$, where (P, P) is the patch resolution and $N = \frac{H \cdot W}{P^2}$ is the resulting sequence length.

We map these patches to a constant latent vector dimension D using a trainable linear projection $\mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}$. We also prepend a learnable embedding $\mathbf{x}_{class} \in \mathbb{R}^D$ whose state at the output of the Transformer encoder serves as the global image representation. To retain positional information, standard 1D position embeddings $\mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D}$ are added:

$$\mathbf{Z}_0 = \left[\mathbf{x}_{class};\ \mathbf{x}_p^1\mathbf{E};\ \mathbf{x}_p^2\mathbf{E};\ \ldots;\ \mathbf{x}_p^N\mathbf{E}\right] + \mathbf{E}_{pos} \tag{44}$$

7.2 The Mathematics of Scaled Dot-Product Attention

The core of the transformer is the Self-Attention mechanism. For a sequence of embeddings $\mathbf{Z} \in \mathbb{R}^{N \times D}$, we project them into Queries ($\mathbf{Q}$), Keys ($\mathbf{K}$), and Values ($\mathbf{V}$) using learned weight matrices $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{R}^{D \times D_k}$.

$$\mathbf{Q} = \mathbf{Z}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{Z}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{Z}\mathbf{W}^V \tag{45}$$

The attention scores are computed as the dot product between every query and every key. Mathematically, the i, j-th entry of the matrix $\mathbf{Q}\mathbf{K}^T$ represents the unnormalized affinity between the i-th token and the j-th token.

Why Scale by $\frac{1}{\sqrt{D_k}}$?

Assume that the components of $\mathbf{q}$ and $\mathbf{k}$ are independent random variables with mean 0 and variance 1. The dot product $q \cdot k = \sum_{i=1}^{D_k} q_i k_i$ has a mean of 0 and a variance determined by:

$$\text{Var}(q_i k_i) = \mathbb{E}[q_i^2 k_i^2] - \mathbb{E}[q_i k_i]^2 \tag{46}$$

$$= \mathbb{E}[q_i^2]\mathbb{E}[k_i^2] - 0 = 1 \cdot 1 = 1 \tag{47}$$

Therefore, the variance of the sum over D_k dimensions is exactly D_k. For large D_k, the dot product grows excessively large in magnitude, pushing the softmax function into regions where gradients are infinitesimally small. Scaling by $\sqrt{D_k}$ restores the variance to 1, ensuring stable gradient flow.

The full attention output is thus:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{D_k}}\right)\mathbf{V} \tag{48}$$

7.3 Multi-Head Attention (MSA) and the MLP Block

Instead of performing a single attention function, we project the queries, keys, and values h times with different, learned linear projections.

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V) \tag{49}$$

$$\text{MSA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^O \tag{50}$$

Within the Transformer Encoder block, the sequence of operations is defined as:

$$\mathbf{Z}'_l = \text{MSA}(\text{LayerNorm}(\mathbf{Z}_{l-1})) + \mathbf{Z}_{l-1} \tag{51}$$

$$\mathbf{Z}_l = \text{MLP}(\text{LayerNorm}(\mathbf{Z}'_l)) + \mathbf{Z}'_l \tag{52}$$

Where the MLP contains two linear layers with a GELU non-linearity:

$$\text{MLP}(x) = \text{GELU}(x\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2 \tag{53}$$

$$\text{GELU}(x) = x\Phi(x) = x \cdot \frac{1}{2}\left[1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right)\right] \tag{54}$$

7.4 Complexity Analysis of ViT

The computational complexity of the Global Self-Attention mechanism is its primary flaw when applied to high-resolution imagery. For an input sequence of length N and dimension D:

- Computing $\mathbf{Q}\mathbf{K}^T$ takes $\mathcal{O}(N^2 D)$ time.
- Computing the Softmax takes $\mathcal{O}(N^2)$ time.
- Multiplying by $\mathbf{V}$ takes $\mathcal{O}(N^2 D)$ time.

Total complexity: $\Omega(\text{MSA}) = \mathcal{O}(N^2 \cdot D)$. For a dermoscopy image of resolution 1024×1024 and patch size 16×16, $N = 4096$. The N^2 term requires processing a 4096×4096 matrix (16.7 million elements) per attention head, per layer, per batch, rendering it computationally infeasible on standard consumer hardware.

8 SWIN TRANSFORMER: HIERARCHICAL SHIFTED-WINDOW MATH

8.1 Window-Based Self Attention (W-MSA) Complexity

Swin (Shifted Window) Transformers alleviate the $\mathcal{O}(N^2)$ bottleneck by restricting self-attention to non-overlapping local windows.

Let the image contain $h \times w$ patches, and each window contain $M \times M$ patches (default $M = 7$). The number of windows is $\frac{h \cdot w}{M^2}$. Within a single window, the sequence length is M^2. The complexity of MSA within one window is therefore $\mathcal{O}((M^2)^2 \cdot C) = \mathcal{O}(M^4 C)$.

Summing across all $\frac{h \cdot w}{M^2}$ windows, the total complexity becomes:

W-MSA Computational Complexity

$$\Omega(\text{W-MSA}) = \left(\frac{h \cdot w}{M^2}\right) \cdot \mathcal{O}(M^4 C) = \mathcal{O}(M^2 \cdot h \cdot w \cdot C) \tag{55}$$

Because M is a fixed constant (7), the computational complexity is now **strictly linear** with respect to the number of image patches $h \cdot w$, matching the efficiency of CNNs.

8.2 Shifted Window Partitioning Mathematics

To enable cross-window connections while maintaining efficiency, Swin introduces the **Shifted Window Partitioning** approach. In layer l, a regular window partitioning scheme is adopted, starting from the top-left pixel. In layer $l + 1$, the window partitioning is shifted by $(\lfloor \frac{M}{2} \rfloor, \lfloor \frac{M}{2} \rfloor)$ pixels from the regular partitioning.

The mathematical formulation for the successive Swin Transformer blocks is:

$$\hat{\mathbf{z}}^l = \text{W-MSA}(\text{LN}(\mathbf{z}^{l-1})) + \mathbf{z}^{l-1} \tag{56}$$

$$\mathbf{z}^l = \text{MLP}(\text{LN}(\hat{\mathbf{z}}^l)) + \hat{\mathbf{z}}^l \tag{57}$$

$$\hat{\mathbf{z}}^{l+1} = \text{SW-MSA}(\text{LN}(\mathbf{z}^l)) + \mathbf{z}^l \tag{58}$$

$$\mathbf{z}^{l+1} = \text{MLP}(\text{LN}(\hat{\mathbf{z}}^{l+1})) + \hat{\mathbf{z}}^{l+1} \tag{59}$$

where W-MSA utilizes standard partitioning, and SW-MSA utilizes shifted partitioning.

8.3 Efficient Batch Computation via Cyclic Shift

The shifted partition creates smaller, irregular windows at the image boundaries. A naive implementation would pad these to $M \times M$ and compute attention, resulting in wasted computation. Swin employs a highly efficient **Cyclic Shift** towards the top-left direction.

Mathematically, let the 2D feature map $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$. The cyclic shift operator S_{cyc} translates the coordinates with wrap-around:

$$\mathbf{X}'_{i,j} = \mathbf{X}_{(i+\lfloor \frac{M}{2} \rfloor) \bmod H,\ (j+\lfloor \frac{M}{2} \rfloor) \bmod W} \tag{60}$$

After shifting, a batched window may contain sub-windows that originated from distant boundaries (e.g., top and bottom edges) which should not attend to each other. A masked self-attention mechanism is utilized. A mask matrix $\mathbf{M} \in \mathbb{R}^{M^2 \times M^2}$ is constructed where $M_{i,j} = 0$ if pixel i and pixel j belong to the same original sub-window, and $M_{i,j} = -\infty$ otherwise.

The attention computation becomes:

$$\text{MaskedAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}} + \mathbf{M}\right)\mathbf{V} \tag{61}$$

Adding $-\infty$ before the softmax effectively zeroes out the attention weight between uncorrelated boundary patches.

8.4 Relative Position Bias Parameterization

Instead of adding absolute position embeddings to the input tokens, Swin computes a **Relative Position Bias** $\mathbf{B} \in \mathbb{R}^{M^2 \times M^2}$ added directly to the attention logits:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}} + \mathbf{B}\right)\mathbf{V} \tag{62}$$

Because the relative position along each axis lies in the range $[-M+1, M-1]$, we parameterize a smaller bias matrix $\hat{\mathbf{B}} \in \mathbb{R}^{(2M-1)\times(2M-1)}$. For two pixels $i = (x_i, y_i)$ and $j = (x_j, y_j)$, their relative coordinate is $\Delta x = x_i - x_j$ and $\Delta y = y_i - y_j$. The specific bias value is extracted by shifting the coordinates to be strictly positive:

$$B_{i,j} = \hat{\mathbf{B}}_{\Delta x + M - 1,\ \Delta y + M - 1} \tag{63}$$

This mathematical property introduces *translation equivariance*, meaning the attention score relies entirely on relative spatial geometry rather than absolute pixel coordinates. For skin cancer detection, where a melanoma can appear anywhere in the frame without reference to the boundary, this property drastically improves generalization.

9 OUR METHODOLOGY & RESULTS

We engineered a robust data pipeline utilizing an **8th class** populated with healthy skin images (Not a Lesion) and applied systematic **minority class oversampling**.

9.1 Loss Formulation: Standard Cross-Entropy

To optimize the network for multi-class classification, we implemented standard Cross-Entropy (CE) loss. This function evaluates the performance of a classification model whose output is a probability distribution across multiple classes, heavily penalizing predictions that diverge from the ground truth labels.

Standard Cross-Entropy Loss

The mathematical formulation for the standard cross-entropy loss for a single sample is:

$$\mathcal{L}_{\text{CE}} = -\log\left(\frac{\exp(z_y)}{\sum_{j=1}^{C}\exp(z_j)}\right) \tag{64}$$

Where:

- C is the total number of classes.
- $y \in \{1, \ldots, C\}$ is the ground truth class index for the given sample.
- z_j represents the raw, unnormalized logit predicted by the network for class j.
- The fraction inside the logarithm represents the Softmax probability p_y assigned to the correct class.

9.2 Two-Phase Fine-Tuning: Linear Algebraic Formulation

To optimize the network without destroying the rich, pre-learned feature representations from the `microsoft/swin-tiny-patch4-window7-224` initialization, we implemented a rigorous two-phase fine-tuning protocol.

Let the pretrained Swin-Tiny backbone be a non-linear mapping $\Phi : \mathbb{R}^{224\times224\times3} \to \mathbb{R}^{768}$, parameterized by the weight matrices Θ_B. Let the newly initialized classification head be $\Psi : \mathbb{R}^{768} \to \mathbb{R}^{8}$ (mapping to our 8 target classes), parameterized by Θ_H. Based on our architecture, the head performs successive affine transformations and non-linearities:

$$\Psi(\mathbf{z}) = \mathbf{W}_2\Big(\text{Dropout}_{0.2}\big(\text{GELU}(\mathbf{W}_1\mathbf{z} + \mathbf{b}_1)\big)\Big) + \mathbf{b}_2 \tag{65}$$

where $\mathbf{z} = \Phi(\mathbf{x}; \Theta_B)$ is the latent feature vector output by the backbone, $\mathbf{W}_1 \in \mathbb{R}^{512\times768}$ projects the features into a dense hidden space, and $\mathbf{W}_2 \in \mathbb{R}^{8\times512}$ maps to the final classification logits.

Phase 1: Head Warm-up (Epochs 1–4)
Initially, the head parameters $\Theta_H = \{\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2\}$ are initialized randomly, while Θ_B contains highly structured weights trained on millions of images. If we were to immediately compute the gradient of the loss $\mathcal{L}$ with respect to the backbone features $\mathbf{z}$, we would apply the chain rule:

$$\nabla_{\mathbf{z}}\mathcal{L} = \left(\frac{\partial \Psi}{\partial \mathbf{z}}\right)^T \nabla_{\hat{\mathbf{y}}}\mathcal{L} \tag{66}$$

Because $\mathbf{W}_1$ and $\mathbf{W}_2$ are entirely random, the Jacobian projection matrix $\frac{\partial \Psi}{\partial \mathbf{z}}$ acts as an unstructured, high-variance scattering matrix. Backpropagating this chaotic, uncalibrated error signal directly into Θ_B would drastically shift the pretrained weights, leading to *catastrophic forgetting* of the Swin Transformer's spatial and texture priors.

To prevent this, Phase 1 strictly enforces $\nabla_{\Theta_B}\mathcal{L} := \mathbf{0}$ by freezing the backbone (`requires_grad = False`). We optimize exclusively with respect to Θ_H using a relatively high learning rate of $\eta_{head} = 10^{-3}$ for 4 epochs, allowing the hyperplanes in $\mathbf{W}_2$ to geometrically align with the latent clusters in $\mathbb{R}^{768}$.

Phase 2: Joint Fine-Tuning (Epochs 5–14)
Once the classifier head is stabilized, we unfreeze Θ_B (`requires_grad = True`) to allow task-specific domain adaptation across the entire network for 10 epochs. To preserve the structural integrity of the backbone, we apply a massive reduction in the learning rate:

$$\Theta_H^{(t+1)} \leftarrow \Theta_H^{(t)} - \eta_{fine_tune}\nabla_{\Theta_H}\mathcal{L} \tag{67}$$

$$\Theta_B^{(t+1)} \leftarrow \Theta_B^{(t)} - \eta_{fine_tune}\nabla_{\Theta_B}\mathcal{L} \tag{68}$$

By setting $\eta_{fine_tune} = 10^{-5}$ (a 100$\times$ reduction relative to the initial head learning rate), we ensure the foundational window-attention filters undergo strict, isomorphic micro-adjustments rather than destructive macroscopic transformations.

9.3 Optimization Dynamics: AdamW with Decoupled Weight Decay

To execute the parameter updates across both phases, we utilized the **AdamW optimizer**. Standard L_2 regularization implemented in native Adam couples the weight decay term directly into the gradient calculation ($g_t = \nabla\mathcal{L} + \lambda\theta_{t-1}$), which inadvertently scales the regularization penalty by the optimizer's historical gradient moments.

AdamW mathematically decouples this operation, ensuring that the true L_2 penalty is applied isothermally to the weights independent of the adaptive learning rate scaling. Using our specific configuration values—weight decay $\lambda = 0.005$ and the current phase's learning rate η—the optimization step proceeds as follows:

AdamW Optimization Algorithm

At time step t, compute gradients $g_t = \nabla\mathcal{L}(\theta_{t-1})$.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \quad \text{(First moment estimate)} \tag{69}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \quad \text{(Second raw moment estimate)} \tag{70}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \quad \text{(Bias corrections)} \tag{71}$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \quad \text{(Decoupled weight decay with } \lambda = 0.005\text{)} \tag{72}$$

Table 2: Final Performance Comparison on 8-Class HAM10000

Model	Train Acc.	Val Acc.	Params	Notes
CNN (from scratch)	70%	62%	∼2M	Severe overfitting
ResNet-50 (transfer)	85%	76%	25.6M	Limited by depth scaling
EfficientNet-B3	90%	82%	11.1M	No global attention mapping
EfficientNet-V2	92%	84%	22M	Better progressive scaling
YOLOv4	88%	79%	–	Bounding box mismatch
Google ViT-B/16	91%	83%	86M	Quadratic complexity limit
Swin-T (Ours)	**97%**	**92%**	29M	**Optimal Global/Local features**

10 CONCLUSION

In this study, we systematically addressed the critical challenges of automated skin lesion classification, navigating the architectural trade-offs between local feature extraction and global context modeling. The inherent limitations of traditional Convolutional Neural Networks (CNNs)specifically their restricted receptive fields and susceptibility to the degradation problemnecessitated a rigorous architectural evolution through ResNet-50, EfficientNet, and standard Vision Transformer baselines.

The culmination of this progression was the deployment of the Swin Transformer. By leveraging the deep mathematical restructuring of the attention mechanism introduced by this architecture, we successfully transitioned from the spatial limitations of convolutional receptive fields to the $\mathcal{O}(M^2 hwC)$ linear efficiency of Shifted Window Self-Attention. This algorithmic efficiency was a pivotal breakthrough, allowing us to process high-resolution 384×384 dermoscopy images effectively without succumbing to the quadratic computational bottlenecks of standard global attention.

Furthermore, the network's hierarchical topology was paired with our custom class-balancing pipeline and a mathematically rigorous Decoupled Weight Decay (AdamW) optimization strategy to explicitly counteract the severe class imbalance inherent in the dataset. This synergistic approach of dynamic global attention and targeted optimization achieved a state-of-the-art

validation accuracy of 92%.

Ultimately, this framework successfully bridges the gap between deep learning theory and practical clinical applicability, providing a highly sensitive, computationally viable, and mathematically sound solution for automated dermatological diagnosis.

11 DATASETS AND REPOSITORIES

The following datasets and online repositories were utilized or referenced in the development and deployment of this skin cancer detection system:

- **HAM10000 Dataset (Kaggle)**: The primary dataset used for training, containing 10,015 dermatoscopic images.
  https://www.kaggle.com/datasets/kmader/skin-cancer-mnist-ham10000
- **ISIC 2020 Dataset (Kaggle)**: Utilized for sourcing the "Not a Lesion" (benign skin) 8th class to balance the dataset.
  https://www.kaggle.com/datasets/abbasashwal/isic-2020-dataset
- **Skin Cancer Dataset (Hugging Face)**: A high-quality dermatoscopic dataset covering major skin lesion types.
  https://huggingface.co/datasets/marmal88/skin_cancer
- **Harvard Skin Dataset (Dataverse)**: A supplementary dataset containing diverse skin lesion images.
  https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/DBW86T
- **ISIC Archive**: The overarching repository for international skin imaging data.
  https://www.isic-archive.com/
  https://challenge2024.isic-archive.com/
- **PAD-UFES-20 Dataset (Mendeley Data)**: Clinical images of skin lesions collected using smartphone devices.
  https://data.mendeley.com/datasets/zr7vgbcyr2/1
- **Skin Lesion Analysis Toward Melanoma Detection (Kaggle)**: An additional repository of ISIC challenge data.
  https://www.kaggle.com/datasets/kmader/skin-lesion-analysis-toward-melanoma-detection
- **Project Frontend Repository**: The GitHub repository hosting the web application frontend.
  https://github.com/JITEN-BHARGA/skin-cancer-frontend.git
- **Live Web Application**: The deployed frontend for the skin cancer detection tool.
  https://skin-cancer-frontend-ffho.vercel.app/

- **Hugging Face Model Space**: The deployed model endpoint.
  https://huggingface.co/spaces/jiten-333/Skin_Cancer

12 REFERENCES

REFERENCES

[1] Tschandl, P., Rosendahl, C., & Kittler, H. (2018). The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions. *Scientific Data*, 5, 180161.

[2] Hasan, M. K., et al. (2023). Review of skin cancer detection and classification methods. *Journal of Biomedical Informatics*.

[3] O'Shea, K., & Nash, R. (2015). An introduction to convolutional neural networks. *arXiv:1511.08458*.

[4] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778.

[5] Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *International Conference on Machine Learning (ICML)*, pp. 6105–6114.

[6] Tan, M., & Le, Q. V. (2021). EfficientNetV2: Smaller models and faster training. *International Conference on Machine Learning (ICML)*, pp. 10096–10106.

[7] Adnan, F., Awan, M. J., et al. (2023). EfficientNetB3-Adaptive Augmented Deep Learning (AADL) for multi-class plant disease classification. *IEEE Access*, 11, 85426–85440.

[8] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788.

[9] Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). YOLOv4: Optimal speed and accuracy of object detection. *arXiv:2004.10934*.

[10] Dosovitskiy, A., et al. (2021). An image is worth 16×16 words: Transformers for image recognition at scale. *International Conference on Learning Representations (ICLR)*.

[11] Zhai, X., Kolesnikov, A., Houlsby, N., & Beyer, L. (2022). Scaling vision transformers. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12104–12113.

[12] Liu, Z., Lin, Y., Cao, Y., et al. (2021). Swin Transformer: Hierarchical vision transformer using shifted windows. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 10012–10022.

[13] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *International Conference on Machine Learning (ICML)*, pp. 448–456.

[14] Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)*.

[15] Huang, G., Sun, Y., Liu, Z., Sedra, D., & Weinberger, K. Q. (2016). Deep networks with stochastic depth. *European Conference on Computer Vision (ECCV)*, pp. 646–661.

[16] Wolchok, J. D., et al. (2017). Overall survival with combined nivolumab and ipilimumab in advanced melanoma. *New England Journal of Medicine*, 377, 1345–1356.